TUGAS 1 - LAPORAN PENGENALAN KOTLIN

NAMA : TOMAS BECKET
NIM : 71230985

A. HELLO, WORLD!

1.
```
fun main(args: Array<String>) {
println("Hello, world!")
}
```

- fun main( ) — awal program kotlin
- Argos: Array<Strings> — parameter untuk menerima argumen
- println (“Hello, world!”) — cetak teks di baris setelahnya

2.
```
fun main() {
println("Hello, world!")
}
```

- fun main( ) — awal program kotlin
- println (“Hello, world!”) — cetak teks di baris setelahnya

3.
```
fun main() = println("Hello, world!")
```

- expression body (=)

Penjelasan :

Ketiga cara diatas menghasilkan output yang sama yaitu `Hello, world!`, hanya saja cara pertama biasanya digunakan jika ingin memahami kotlin tapi mirip dengan java, cara kedua sama persis dengan yang cara pertama cuma lebih ringkas, cara ketiga masih menggunakan fungsi main yang sama tapi menggunakan expression body sehingga menjalankan satu expresi saja di 1 line atau baris.

B. THE BASICS

Kode :
```
fun main(args: Array<String>) {
print("Hello")
println(", world!")
}
```

Output :
```
Hello, world!
```

Penjelasan : dari kode diatas menunjukan bahwa main merupakan fungsi utama pada kotlin yang pertama kali dijalankan, Array<String> menerima argumen bertipe string, dan print("Hello") mencetak teks tanpa pindah baris, lalu println(", world!") mencetak teks lanjutannya dan menambahkan di baris baru, sehingga output yang muncul hanya satu baris

C. VARIABLES

Kode :

```
fun main() {
    val a: Int = 1      // tidak bisa diubah
    var b = 2           // bisa diubah

    b = a               // b = 1

    val c: Int
    c = 3               // assign sekali, tidak bisa diubah lagi

    println("a = $a")
    println("b = $b")
    println("c = $c")
}
```

Output :

```
a = 1
b = 1
c = 3
```

Penjelasan : val dan var mendeklarasikan sebuah variabel pada kotlin. var bersifat mutable, sedangkan val bersifat immutable. Sehingga sebagai contoh val a: Int = 1 adalah val yang langsung diberi nilai dan tidak bisa diubah lagi, var b = 2 otomatis bertipe Int dan nilainya boleh diubah seperti b = a, sedangkan val c: Int boleh diisi belakangan dengan c = 3 tapi setelah itu tidak bisa diubah lagi

Kode :

```
fun main() {
    val a: Int = 1      // tidak bisa diubah
    var b = 2           // bisa diubah

    b = a               // b = 1

    val c: Int
    c = 3               // assign sekali, tidak bisa diubah lagi
    a = 4               // Error: Val gak bisa di reassigned

    println("a = $a")
    println("b = $b")
    println("c = $c")
}
```

Output :

```
'val' cannot be reassigned.
```

Penjelasan : baris a = 4 akan menghasilkan error karena a adalah val yang tidak boleh di reassign.

Kode :

```
const val NAME = "Kotlin"

fun main() {
    val nameLowered = NAME.lowercase()

    println(NAME)
    println(nameLowered)
}
```

Output :

```
Kotlin
kotlin
```

Penjelasan : const val NAME = "Kotlin" merupakan konstanta compile-time, artinya nilainya sudah pasti, sedangkan val nameLowered = NAME.lowercase() adalah variabel biasa yang nilainya baru dihitung.

D. FUNCTIONS

Kode :

```
fun sum(a: Int, b: Int): Int {
    return a + b
}

fun mul(a: Int, b: Int) = a * b

fun printMul(a: Int, b: Int): Unit {
    println(mul(a, b))
}

fun printMul1(a: Int = 1, b: Int) {
    println(mul(a, b))
}

fun printMul2(a: Int, b: Int = 1) = println(mul(a, b))

fun main() {
    println(sum(1, 2))
    printMul(2, 2)
    printMul1(b = 2)
    printMul2(5)
}
```

Output :

```
3
4
2
5
```

Penjelasan : kode diatas merupakan single expression function yang dimana ada penggunaan return untuk mengembalikan nilai, = expression bentuk singkat dari fungsi, Unit artinya fungsi yang tidak mengembalikan nilai, dan terdapat Default parameter seperti (b: Int =1), juga merupakan type inference artinya kotlin bisa menebak tipe data secara otomatis.

E. IF EXPRESSION

Kode :

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) {
        return a
    } else {
        return b
    }
}

fun main() {
    println(maxOf(3, 7))
}
```

==

```
fun maxOf(a: Int, b: Int) =
if (a > b) {
    a
} else {
    b
}


fun main() {
    println(maxOf(3, 7))
}
```

Output :

```
7
```

Penjelasan : if bisa dipakai sebagai expression. Fungsi maxOf(a: Int, b: Int): Int menerima dua angka dan mengembalikan mana yang paling besar di dalamnya, kondisi if (a > b) dicek, jika benar maka return a, kalau salah maka return b. Secara konsep, if else di sini sebenarnya menghasilkan sebuah nilai (entah a atau b), lalu nilai itu dikembalikan oleh fungsi, sehingga fungsi ini selalu mengembalikan satu Int, yaitu nilai maksimum dari a dan b

F. WHEN EXPRESSION

Kode :

```
fun main() {
    val x = 2

    when (x) {
        1 -> print("x = 1")
        2 -> print("x = 2")
        else -> print("x is neither 1 nor 2")
    }
}
```

Output :

```
x = 2
```

Penjelasan : when di Kotlin merupakan struktur percabangan, bisa mengecek nilai langsung seperti when (x) untuk membandingkan nilai tertentu (misalnya 1, 2, dll.), atau mengecek kondisi seperti when { x < 0 -> ... } yang mirip if else. Jika tidak ada kondisi yang cocok, bagian else akan dijalankan. Selain itu, when bisa digunakan sebagai expression, artinya bisa menghasilkan nilai dan disimpan ke dalam variabel, sama seperti if di Kotlin.

G. WHEN STATEMENT

Kode :

```
fun main() {
    val teaSack = "Oolong"

    when (teaSack) {
        "Oolong" -> error("We don't serve Chinese tea like $teaSack!")
        "Trial", "Uncertified" ->
            error("We cannot serve uncertified tea!")
    }

    println("Tea is served")
}
```

Output :

```
Exception in thread "main" java.lang.IllegalStateException: We don't serve
Chinese tea like Oolong!
   at FileKt.main (File.kt:5)
   at FileKt.main (File.kt:-1)
   at jdk.internal.reflect.NativeMethodAccessorImpl.invoke0 (:-2)
```

Kode :

```
fun main() {
    val teaSack = "Green"
```

Output :

```
Tea is served
```

Penjelasan : when statement dipakai untuk memilih aksi berdasarkan kondisi/nilai, bisa punya banyak pilihan dalam satu cabang, dan else tidak wajib kalau hanya dipakai sebagai statement.

H. && VS AND

Kode :

```
fun checkB(): Boolean {
    println("checkB() dipanggil")
    return true
}

fun main() {
    val a = false

    println("Pakai && :")
    if (a && checkB()) {
        println("Masuk if &&")
    }

    println("Pakai and :")
    if (a and checkB()) {
        println("Masuk if and")
    }
}
```

Output :

```
Pakai && :
Pakai and :
checkB() dipanggil
```

Penjelasan : Di Kotlin, && dan || adalah operator logika yang memakai short-circuit evaluation, artinya pada ekspresi a && b jika a sudah bernilai false maka b tidak dicek, dan pada a || b jika a sudah bernilai true maka b juga tidak dicek. Berbeda dengan itu, and dan or adalah fungsi yang tidak menggunakan short-circuit, sehingga pada a and b maupun a or b kedua sisi ekspresi akan selalu dievaluasi, meskipun hasil akhirnya sebenarnya sudah bisa ditentukan dari sisi pertama.

I. LOOPS

Kode :

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")

    var index = 0
    while (index < items.size) {
        println("item at $index is ${items[index]}")
        index++
    }
}
```

Output :

```
item at 0 is apple
item at 1 is banana
item at 2 is kiwifruit
```

Penjelasan : loops atau perulangan digunakan untuk menjalankan blok kode yang sama berulang kali selama suatu kondisi masih terpenuhi. Contohnya, while akan mengecek kondisi sebelum menjalankan kode, jadi kalau kondisinya sejak awal salah, kodenya bisa tidak dijalankan sama sekali, sedangkan do...while akan menjalankan kode terlebih dulu minimal satu kali baru kemudian mengecek kondisi, sehingga cocok dipakai kalau jika ingin memastikan proses berjalan setidaknya sekali.

J. RANGES

Kode :

```
fun main() {
    val x = 10
    if (x in 1..10) {
        println("fits in range")
    }

    for (i in 1..5) {
        print(i)
    }
    println()

    for (i in 9 downTo 0 step 3) {
        print(i)
    }
}
```

Output :

```
fits in range
12345
9630
```

Penjelasan : Ranges mengecek apakah nilai ada di antara dua angka dan membuat perulangan angka dengan cara yang rapi dan mudah dibaca

K. NULL SAFETY

Kode :

```
val notNullText: String = "Definitely not null"
val nullableText1: String? = "Might be null"
val nullableText2: String? = null

fun funny(text: String?) {
    if (text != null)
        println(text)
    else
        println("Nothing to print :(")
}

fun funnier(text: String?) {
    val toPrint = text ?: "Nothing to print :("
    println(toPrint)
}

fun main() {
    funny(notNullText)
    funny(nullableText1)
    funny(nullableText2)

    funnier(nullableText1)
    funnier(nullableText2)
}
```

Output :

```
Definitely not null
Might be null
Nothing to print :(
Might be null
Nothing to print :(
```

Penjelasan : Null Safety di Kotlin adalah fitur yang mencegah error karena nilai null dengan cara memisahkan tipe yang boleh null dan yang tidak boleh null. Tipe tanpa ? (misalnya String) tidak bisa bernilai null, sedangkan tipe dengan ? (misalnya String?) boleh bernilai null, sehingga kita dipaksa untuk mengecek atau menangani null secara eksplisit, misalnya dengan if (x != null) atau dengan Elvis operator ?:, supaya program lebih aman dan tidak mudah crash karena NullPointerException.

L. ELVIS OPERATOR ?:

Kode :

```
fun findItem(id: String): String? {
    return if (id == "1") "Data ditemukan" else null
}

fun loadInfoById(id: String): String? {
    val item = findItem(id) ?: return null
    return item.lowercase() ?: throw Exception("Gagal load info")
}

fun main() {
    println(loadInfoById("1"))
    println(loadInfoById("2"))
}
```

Output :

```
data ditemukan
null
```

Penjelasan : Elvis operator ?: di Kotlin digunakan untuk menangani nilai null dengan cara singkat. Cara kerjanya: jika ekspresi di sebelah kiri tidak null, maka nilai itu yang dipakai; tetapi jika null, maka yang dipakai adalah ekspresi di sebelah kanan. Selain itu, bagian kanan hanya akan dijalankan jika sisi kiri memang null, sehingga aman dan efisien.

M. SAFE CALLS

Kode :

```
data class Person(val name: String?)
data class Department(val head: Person?)
data class Employee(val department: Department?)

fun printDepartmentHead(employee: Employee) {
    println(employee.department?.head?.name)
}

fun main() {
    val head1 = Person("Andi")
    val dept1 = Department(head1)
    val emp1 = Employee(dept1)

    printDepartmentHead(emp1)
    emp1.department?.head?.name?.let {
        println("Head name with let: $it")
    }

    val emp2 = Employee(null)

    printDepartmentHead(emp2)
    emp2.department?.head?.name?.let {
        println("This will not be printed: $it")
    }
}
```

Output :

```
Andi
Head name with let: Andi
null
```

Penjelasan : Safe Call (?.) di Kotlin, yaitu cara aman mengakses properti/objek yang bisa null tanpa bikin program crash (NullPointerException). Kalau pakai someThing?.otherThing, maka kalau someThing itu null, ekspresi langsung berhenti dan hasilnya null, bukan error. safe call enak dipakai kalau chaining, misalnya employee.department?.head?.name: kalau salah satu di tengah bernilai null (department atau head), Kotlin tidak lanjut dan hasil akhirnya null, tapi program tetap jalan.

N. UNSAFE CALLS

Kode :

```kotlin
data class Person(val name: String?)
data class Department(val head: Person?)
data class Employee(val department: Department?)

fun printDepartmentHead(employee: Employee) {
    // Unsafe call dengan !!
    println(employee.department!!.head!!.name!!)
}

fun main() {
    val emp = Employee(null)   // department = null
    printDepartmentHead(emp)
}
```

Output :

```
Exception in thread "main" java.lang.NullPointerException
   at FileKt.printDepartmentHead (File.kt:7)
   at FileKt.main (File.kt:12)
   at FileKt.main (File.kt:-1)
```

Penjelasan : Unsafe call (!!) di Kotlin merupakan operator yang memaksa nilai nullable dianggap tidak null, tetapi jika ternyata nilainya null, program akan langsung crash dengan NullPointerException, sehingga berbahaya dan sebaiknya dihindari, dan lebih aman memakai ?., ?:, atau let untuk menangani nilai null.

O. TODO

Kode :

```kotlin
data class Item(val id: String)

fun findItemOrNull(id: String): Item? = TODO("Find item $id")

fun main() {
    findItemOrNull("A01")
}
```

Output :

```
Exception in thread "main" kotlin.NotImplementedError: An operation is not implemented:
Find item A01
   at FileKt.findItemOrNull (File.kt:3)
   at FileKt.main (File.kt:6)
   at FileKt.main (File.kt:-1)
```

Penjelasan : TODO() di Kotlin adalah fungsi bawaan yang dipakai sebagai penanda kode yang belum diimplementasikan. Program tetap bisa dikompilasi, tapi kalau fungsi itu dipanggil saat runtime, aplikasi langsung error dengan NotImplementedError.

P. STRING TEMPLATES AND THE STRING BUILDER

Kode :

```
fun main() {
    val i = 10
    val s = "Kotlin"

    println("i = $i")
    println("Length of $s is ${s.length}")
}
```

Output :

```
i = 10
Length of Kotlin is 6
```

Penjelasan : String Templates dipakai untuk menyisipkan nilai variabel atau ekspresi langsung ke dalam string tanpa perlu pakai +

Q. LAMBDA EXPRESSIONS

Kode :

```
fun main() {
    val sum: (Int, Int) -> Int = { x: Int, y: Int -> x + y }
    val mul = { x: Int, y: Int -> x * y }

    println(sum(3, 4))
    println(mul(3, 4))
}
```

Output :

```
7
12
```

Penjelasan : Lambda expression adalah fungsi tanpa nama yang bisa disimpan ke dalam variabel atau langsung dikirim sebagai parameter ke fungsi lain. Bentuk contohnya seperti ini: { parameter -> isi_kode }. Karena Kotlin mendukung *functional programming*, lambda sering dipakai untuk operasi singkat seperti penjumlahan, perkalian, filter data, dan sebagainya.